Effective Agentic Coding for Scientific Computing

Coding is the easy part now: asking AI the right questions, checking its answers, and training researchers to do this is what's hard

---

James Kermode · School of Engineering, University of Warwick

27 October 2026 · CECAM Mary Ann Mansigh Lecture (online)

The end of human programming?

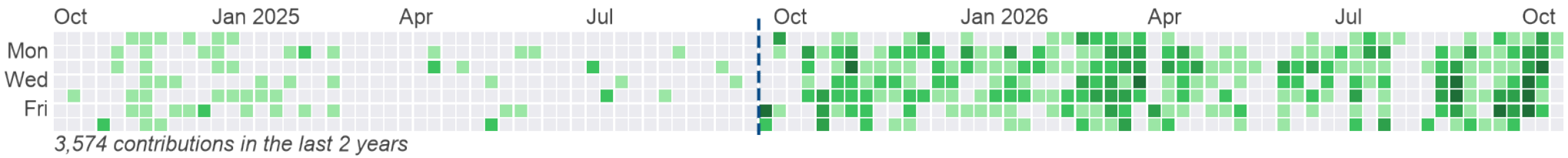
> *“… code generation now happening so quickly that human review is impossible. This forces us to rely on AI itself for validation and regression testing.”*
>
> — G. Kresse, abstract for this afternoon

Other talks have emphasised that validation is what separates plausible physics from correct physics, and that it remains the scientist’s job.

Whether tests are implemented by humans or AI, someone has to know what to validate against and which question to ask.

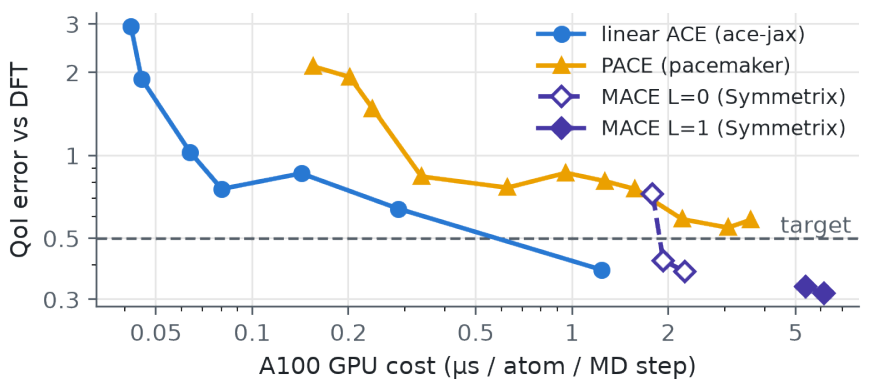
Who will be able to do that in ten years’ time?

My experience of agentic coding: ace-jax



2,332 Claude co-authored commits, across 52 repositories (Oct 2025 – Oct 2026).

- JAX port of ACE interatomic potentials, CPU and GPU. Available via `pip install ace-jax`
- Agent co-authored ≈20k lines of source, ≈28k of tests
- Parity ensured in CI against ACEpotentials.jl and ML-PACE / python-ace, at 10^{-12}–10^{-14}
- **Human work:** high-level design (e.g. choice of frameworks and algorithms), choosing reference systems, tolerances, benchmarks, certifying result



Bug: basis columns misaligned. Every test passed: the Julia parity CI checked the evaluator on a model Julia built, not on ace-jax's own basis builder.

Fix and post-mortem: built-basis parity against Julia (agrees to 10^{-14}; without the fix, most blocks fail), and missing reference data now fails instead of skipping.

Data: GAP-18 (Bartók et al., Phys. Rev. X, 2018). Parity reference: ACEpotentials.jl (Witt et al., J. Chem. Phys. 2023). LAMMPS: lammps-jax (Gangan, Witt & Kotak, 2026). Compared: pacemaker (Bochkarev et al., Phys. Rev. Mater. 2022); MACE (Batatia et al., NeurIPS 2022) run by Symmetrix (W. C. Witt).

github.com/ACEsuit/ace-jax · acesuit.github.io/ace-jax · benchmarks

Benchmark task: once fully specified, difficulty collapses

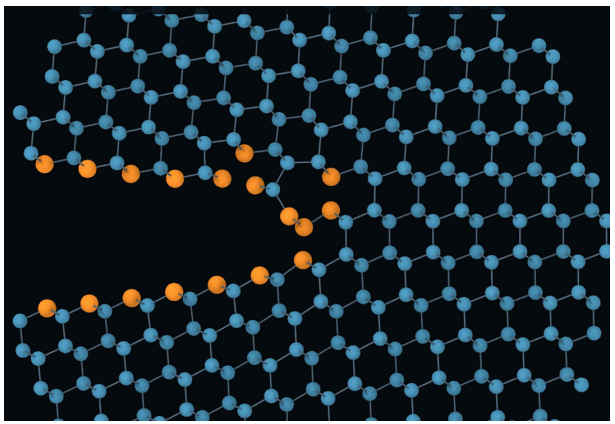
Terminal-Bench-Science (TBS): my contributed `si-fracture-fbc` task. Si(111) crack tip, Stillinger–Weber, flexible continuum linear elastic boundary + Cauchy–Born corrector. Implement from scratch and locate lattice-trapping folds; 16–24 h expert estimate

Version	Change	Result
v0.1 (Aug)	original task	previous AI model generation: no valid pass
Repaired	corrected a bug in reference	GPT-6 Astra: passed, 21 min, $6
Hardened	mixed-mode $K_{\mathrm{I}}/K_{\mathrm{II}}$ crossover	Astra: passed, 65 min
Hardened again	reconstructed-branch fold	Opus 5.5 ✓, Astra ✓, ≈1 h, $7–13
3D follow-up	crack-front kinks, free energy at 600 K (≈40 h expert task)	Opus 5.5 ✓, ≈1.5 h, key physics unprompted

AI Labs now headline with science benchmarks like TBS

Opus 5 29% → Opus 5.5 59%; GPT-5.6 Sol 22% → GPT-6 Astra 65%

terminal-bench-science.ai · task: si-fracture-fbc



Reference solution at the upper fold (Stillinger–Weber), the last trapped state before the crack advances one lattice period. Gold: under-coordinated atoms.

Once a task is fully specified, the difficulty collapses. The human work is choosing the scientific question and validating results.

Where did we learn to do this?

Asking the question

TBS benchmark: specification is now the hard part, implementation is not.

Deciding what counts as correct

ace-jax: consistency checks are not correctness checks. Green CI ≠ right answer.

These skills came from struggle: years of debugging, deriving, and being wrong.

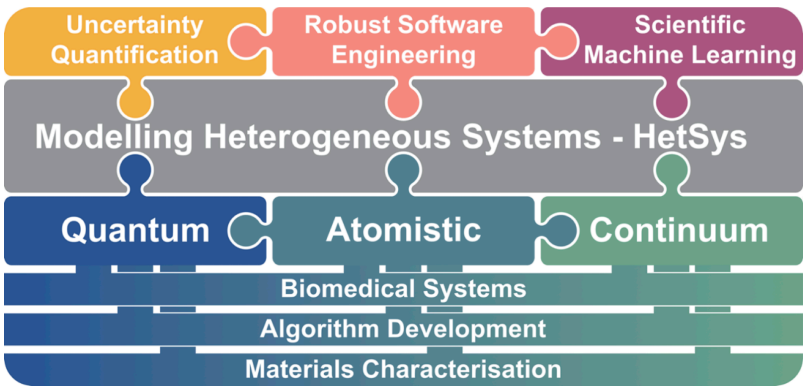
Without the ‘struggle-as-pedagogy’ of a traditional PhD, they will not be automatic.

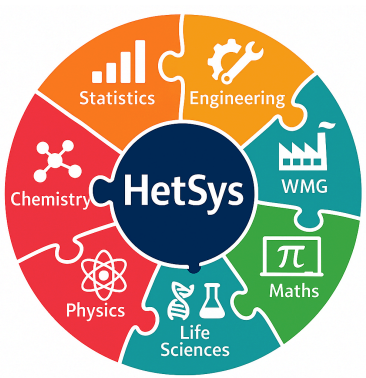
This doesn’t mean preventing new researchers from using these tools at all, but rather deciding deliberately which friction is educational and which can be removed without loss.

HetSys Centre for Doctoral Training AI policy

EPSRC Centre for Doctoral Training in Modelling of Heterogeneous Systems at Warwick, spanning physical sciences with an emphasis on computational modelling. New AI policy for this academic year.

- Skill erosion is the central concern
- Principles (adapted from DTU Physics and Vancouver Guidelines): all content is your responsibility, AI is not an author, and substantive use must be declared
- Build foundations by hand in PhD years 1–2: “the friction is educational”
- Keep one regular problem you solve without AI
- Tests use numerical tolerances you choose, not the AI
- Avoid “verification theatre”: code looks verifiable but isn’t
- Declare substantive use: supervisors too, to normalise
- Everyday test: would the same help from a colleague be acceptable?





Assessing understanding, not typing of code

Not all MSc/PhD code will be human-written. Assessment should separate what a machine can check from what a human must.

Formative: code

- Automated checks run as each cell is completed
- Partial credit and targeted feedback, fast turnaround
- Low stakes

Summative: understanding

- **Random mini-vivas**: 10–15 min discussing the submitted code with marker; sample allocated after submission
- However it was written, they have to defend it

Delegate pattern-matching 'did the code produce the right answer?' to the machine, keeping the human question 'did the student understand?' for humans.

Patterns and Anti-patterns for Agentic Scientific Computing

Patterns that work

- Hoard things you know how to do
- Spec first, implement second
- Red/green TDD
- Manage context obsessively

Anti-patterns

- Unreviewed code on collaborators
- Trusting output you can’t verify
- Stateful notebooks
- Over-engineering by default

Full talk, with the evidence for each:
github.com/jameskermode/agentic-coding-seminar

Discussion

Who will train the researchers of the future,
if we outsource all coding to agents now?

Full talk, slides + source

github.com/jameskermode/agentic-coding-seminar

Acknowledgements: Claude usage supported by credits from Anthropic's AI for Science program. HetSys is funded by EPSRC (EP/S022848/1, EP/Y035429/1). Thanks to Karsten Jacobsen (DTU Physics) for sharing AI guidelines that were starting point for HetSys policy, and to the HetSys staff and students who co-created it. Thanks to TBS team and reviewers. Benchmarks ran on Sulis (EPSRC EP/T022108/1, HPC Midlands+). Claude Code used for slide design; all opinions my own and evidence checked.

UKRI Engineering and Physical Sciences Research Council